Transformer Architecture

A Visual Step-by-Step Guide

"Attention Is All You Need" — Vaswani et al., 2017

This guide covers every major component of the original Transformer with annotated diagrams and mathematical formulations: Input Embedding & Positional Encoding · Q/K/V Projections · Scaled Dot-Product Attention · Multi-Head Attention · Add & Norm · Feed-Forward Network · Encoder Block · Autoregressive Decoding · Output Projection · Training (Loss & Optimisation).

Step 1 — Computing Q, K, V from Input X

What is happening?

The input matrix **X** (seq_len × d_model) is linearly projected into three separate spaces to form Query (Q), Key (K), and Value (V). Each uses its own independently learned weight matrix, so Q/K/V encode different aspects of the sequence even though the same X enters all three multiplications.

Symbol	Shape	Role
X	seq_len × d_model	Token embeddings + Positional Encoding
W_Q	d_model × d_model	Learned weight → Query space
W_K	d_model × d_model	Learned weight → Key space
W_V	d_model × d_model	Learned weight → Value space
Q = X·W_Q	seq_len × d_k	Query — 'What am I looking for?'
K = X·W_K	seq_len × d_k	Key — 'What do I offer?'
V = X·W_V	seq_len × d_v	Value — 'What information do I carry?'

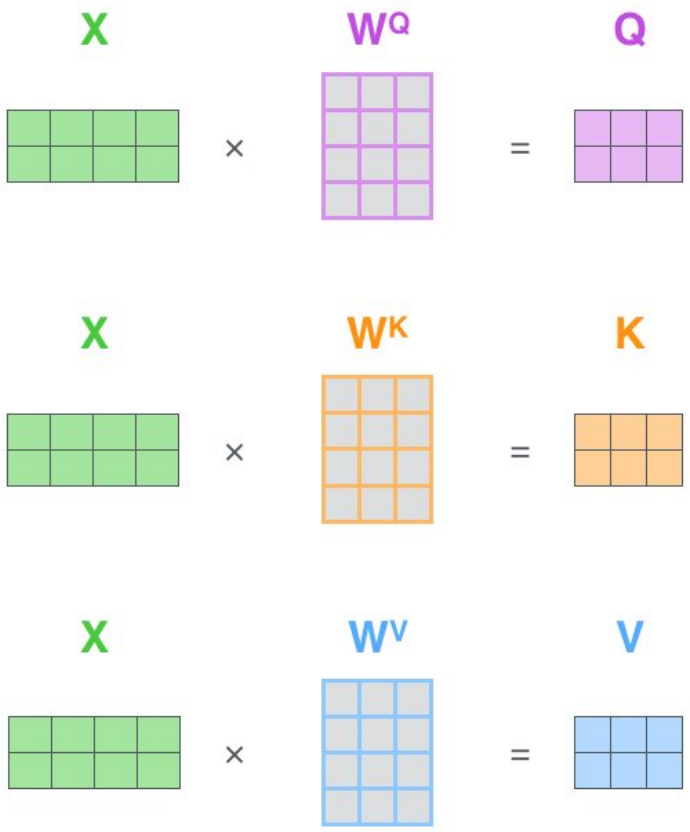

Figure 1 — Same X multiplied by three distinct weight matrices (W_Q, W_K, W_V) to produce Q, K, V. Role separation emerges through backpropagation.

Step 2 — Scaled Dot-Product Attention

What is happening?

Given Q, K, V, the mechanism computes a weighted sum of Values where weights reflect how relevant each Key is to a given Query. Output Z has the same shape as V.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Sub-step	Operation	Why
①	Q · K^T → score (seq×seq)	Dot-product measures pairwise similarity. High score = aligned query & key.
②	÷ sqrt(d_k)	Prevents large dot-products in high dimensions that push softmax into near-zero gradient regions.
③	softmax(scaled scores)	Normalises each row to a probability distribution (sum=1). Determines attention weights.
④	weights · V → Z	Weighted sum of Values. Each output is a blend of all Values by attention distribution.

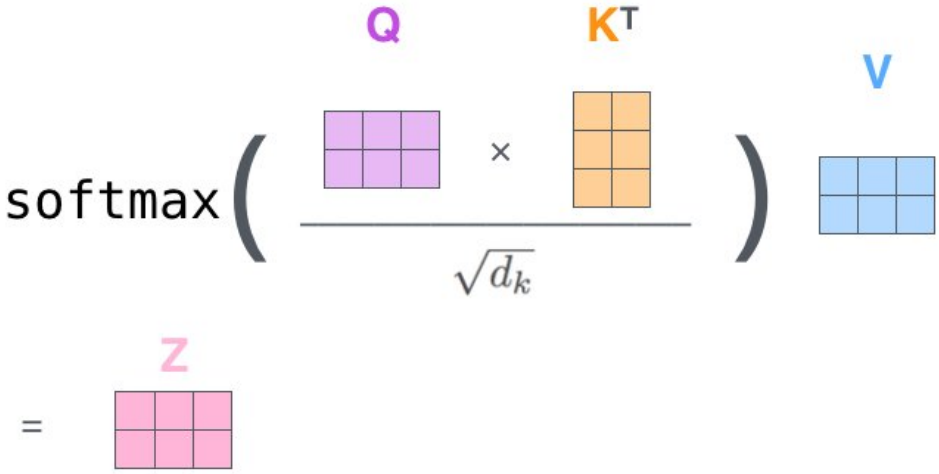

Figure 2 — Scaled Dot-Product Attention. Q·K^T is scaled by sqrt(d_k), softmax gives attention weights, then multiplied by V to produce Z.

Step 3 — Multi-Head Attention

What is happening?

h=8 parallel attention heads each operate on a different learned projection of Q,K,V (d_k = d_model/h = 64 per head). Outputs are concatenated and projected via W_O. Allows the model to attend to multiple representation subspaces simultaneously — e.g. syntax in one head, semantics in another.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

Symbol	Shape	Meaning
h	8	Number of parallel attention heads
d_k = d_v	d_model/h = 64	Dimension per head
W_Q_i,W_K_i,W_V_i	d_model × d_k	Per-head learned projections
head_i	seq_len × d_v	Attention output of head i
Concat	seq_len × d_model	All heads concatenated
W_O	d_model × d_model	Final linear projection
Z	seq_len × d_model	Multi-head attention output

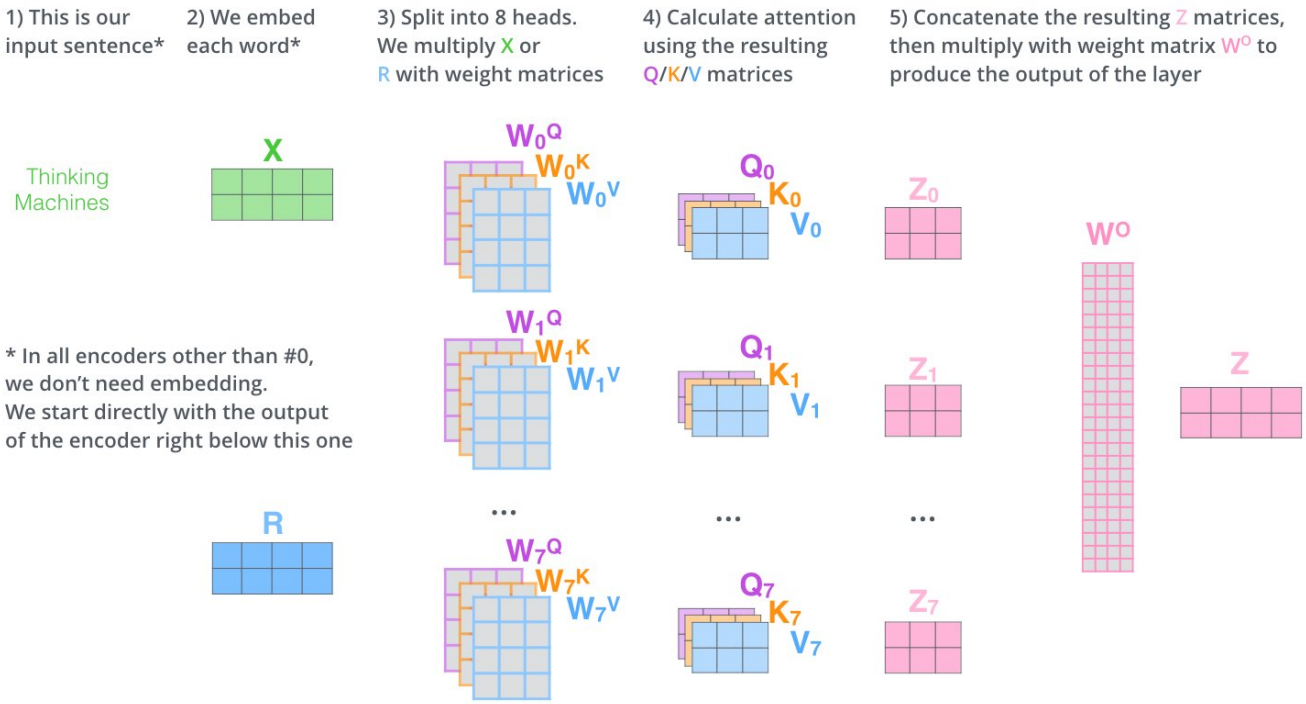

Figure 3 — Each head i computes Q_i, K_i, V_i with independent weights, runs scaled dot-product attention, then all Z_i are concatenated and projected through W_O to produce the final Z.

Step 4 — What Attention Heads Learn (Visualisation)

What is happening?

After training, each head learns distinct linguistic patterns. The heatmap below (Layer 5, from the paper) shows that the word *'it'* correctly attends to *'animal'* — resolving pronoun coreference over a long range.

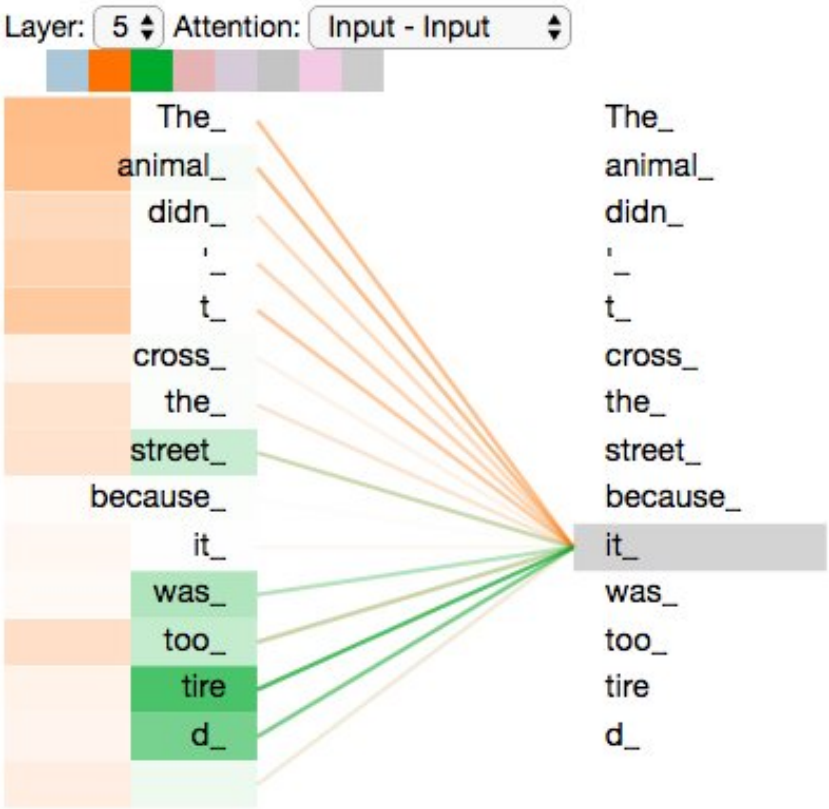

Figure 4 — Attention weights for 'it'. Line thickness = attention strength. Different colours = different heads. The model resolves 'it' → 'animal'.

Step 5 — Positional Encoding

What is happening?

Transformers process all tokens in parallel and have no inherent notion of order. Positional Encoding (PE) injects position information by adding a fixed vector to each token embedding.

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

$$\mathrm{PE}(\mathrm{pos}, 2i{+}1) = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

$$\mathbf{x}_{\mathrm{in}} = \mathrm{Embedding}(x) \times \sqrt{d_{\mathrm{model}}} + \mathrm{PE}$$

Symbol	Meaning
pos	Token position index (0, 1, 2, …)
i	Dimension index (0 … d_model/2–1)
2i	Even dimensions → sine (slow-to-fast oscillation)
2i+1	Odd dimensions → cosine (slow-to-fast oscillation)
10000	Base frequency constant controlling wavelength range

Why sin/cos? ① Bounded in [–1,1] — PE never dominates the embedding. ② Every position is unique. ③ PE(pos+k) is a linear transform of PE(pos) for any fixed k — the model implicitly learns relative distances. ④ Computed analytically → handles sequences longer than seen during training.

Embeddings are scaled by sqrt(d_model) before addition so the embedding signal is not overwhelmed by PE (which is always O(1)).

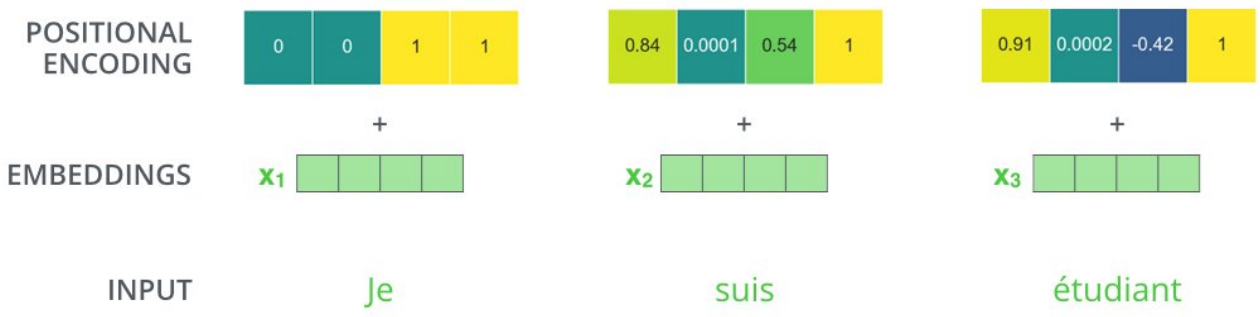

Figure 5 — PE values for positions 1, 2, 3 (d_model=4 example). Each token's final input = embedding + PE.

Step 6 — Encoder Block: Add & Norm (Layer Normalisation)

What is happening?

After each sub-layer (Self-Attention or FFN), the Encoder applies a **Residual Connection** followed by **Layer Normalisation**: LayerNorm(x + Sublayer(x)).

Residual Connection

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Adding x back ensures gradients can flow directly through deep stacks (N=6 layers), preventing vanishing gradients.

Layer Normalisation

$$\hat{x} = \frac{x - \mu}{\sigma + \varepsilon}, \qquad \text{LayerNorm}(x) = \gamma \hat{x} + \beta$$

Symbol	Shape	Meaning
x	seq_len×d_model	Input to LayerNorm (after residual add)
μ, σ	seq_len×1	Mean and std computed across d_model per token
ε	≈1e-6	Numerical stability constant
γ (gamma)	d_model	Learned scale — initialised to 1
β (beta)	d_model	Learned shift — initialised to 0

Normalisation is applied within each token across d_model (not across the batch), making it independent of batch size and stable for variable-length sequences.

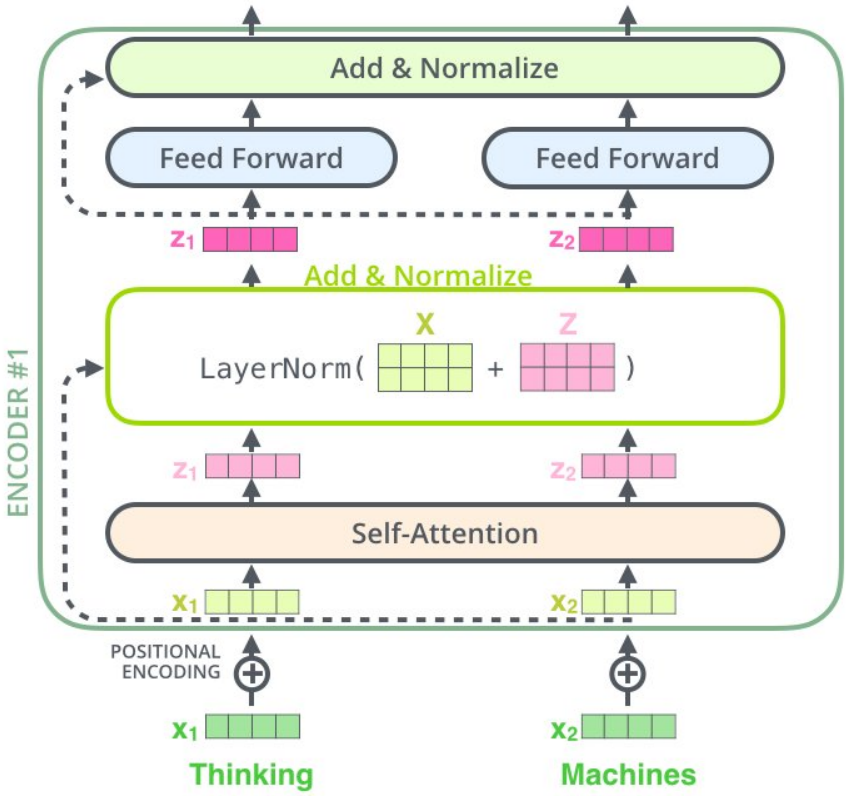

Figure 6 — Encoder Block. Self-Attention outputs z_1,z_2. Residual adds x_1,x_2 back, then LayerNorm is applied. Same Add&Norm; wraps the FFN sub-layer.

Step 7 — Feed-Forward Network (FFN)

What is happening?

After Multi-Head Attention + Add&Norm, each position passes independently through a two-layer FFN. It expands to d_ff=2048 (4× d_model), applies ReLU, then projects back. This gives the model capacity to transform representations after attention has mixed information across positions.

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\,W_2 + b_2$$

Symbol	Shape	Meaning
x	seq_len×d_model	Input from previous Add&Norm;
W1, b1	d_model→d_ff	First linear layer (512→2048)
max(0,·)	—	ReLU activation — non-linearity
W2, b2	d_ff→d_model	Second linear layer (2048→512)
output	seq_len×d_model	Same shape as input; passed to next Add&Norm;

The FFN acts as a per-position key-value memory: W1 rows pattern-match the input (keys), W2 columns retrieve learned associations (values).

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\,W_2 + b_2$$

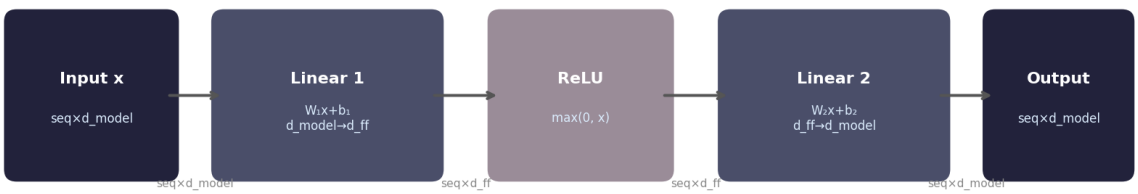

Figure 7 — FFN data flow. Each token independently traverses Linear(d_model→d_ff) → ReLU → Linear(d_ff→d_model). Paper: d_model=512, d_ff=2048.

Step 8 — Decoder: Autoregressive Decoding

What is happening?

The Decoder generates one token at a time. At each step t it receives: (a) the Encoder's fixed output (K_encdec, V_encdec), and (b) all tokens generated so far (shifted right, starting with [START]). It outputs a probability over the vocabulary and selects the next token. Repeats until [END] is emitted.

Step 1 — Encoder processes full source; Decoder sees only [START].

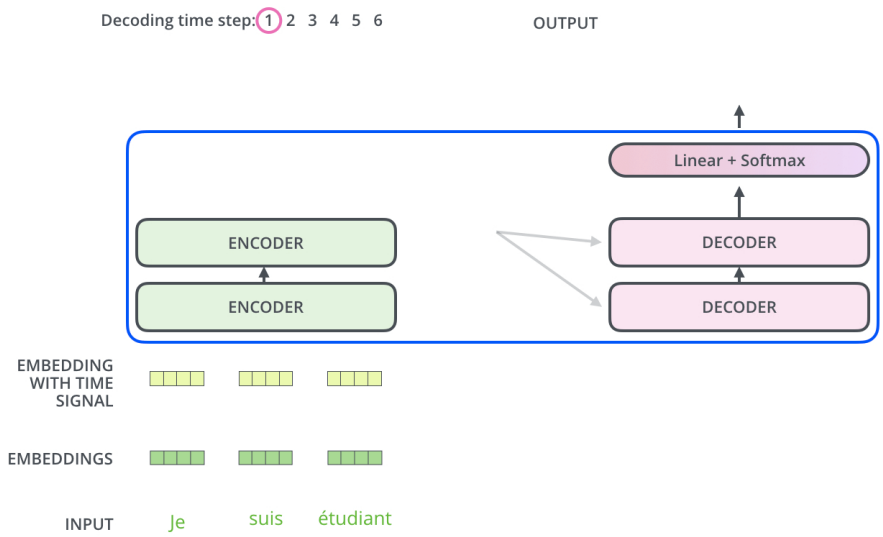

Figure 8a — Decoding step 1. Encoder (green) runs once; Decoder (pink) starts from [START] token.

Step 2 — First output token fed back; Cross-Attention queries encoder.

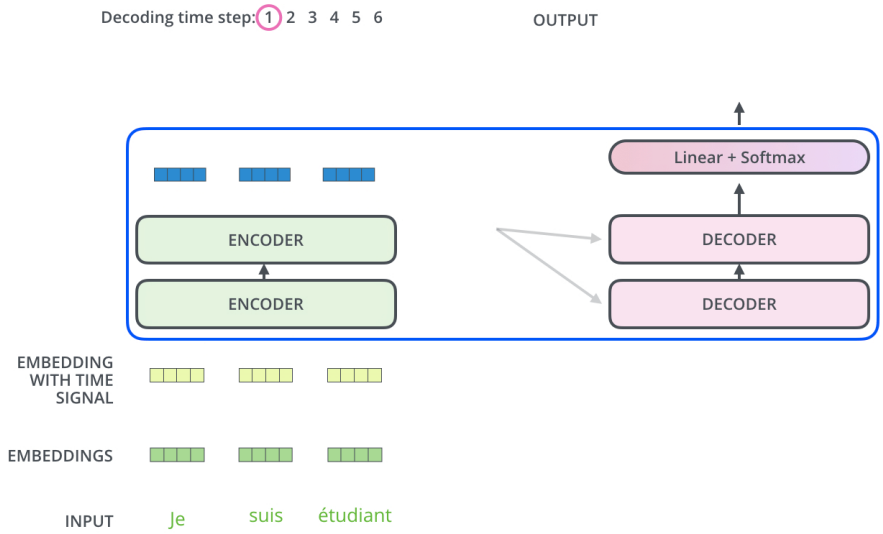

Figure 8b — Decoding step 2. Previous output appended. K_encdec/V_encdec are reused from encoder at every step.

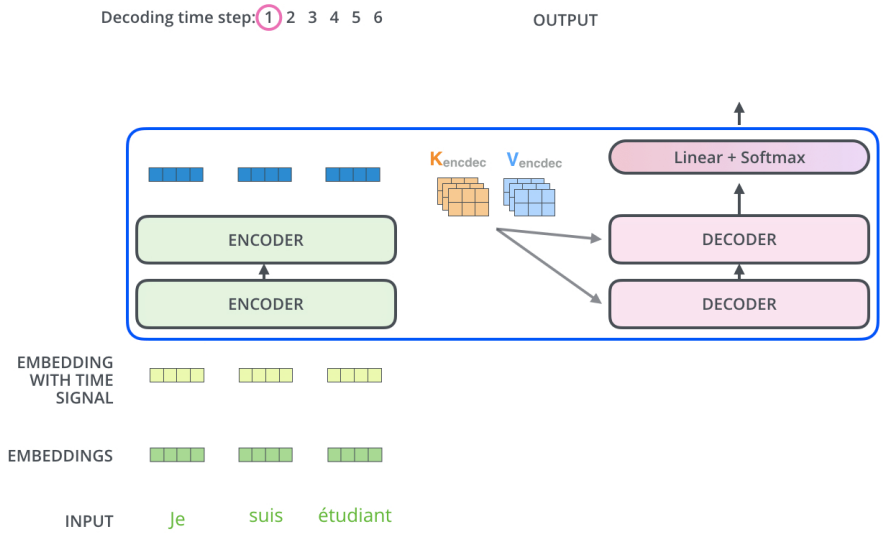

Figure 8c — Each new token is conditioned on all previous outputs (masked self-attention) and full encoder context (cross-attention).

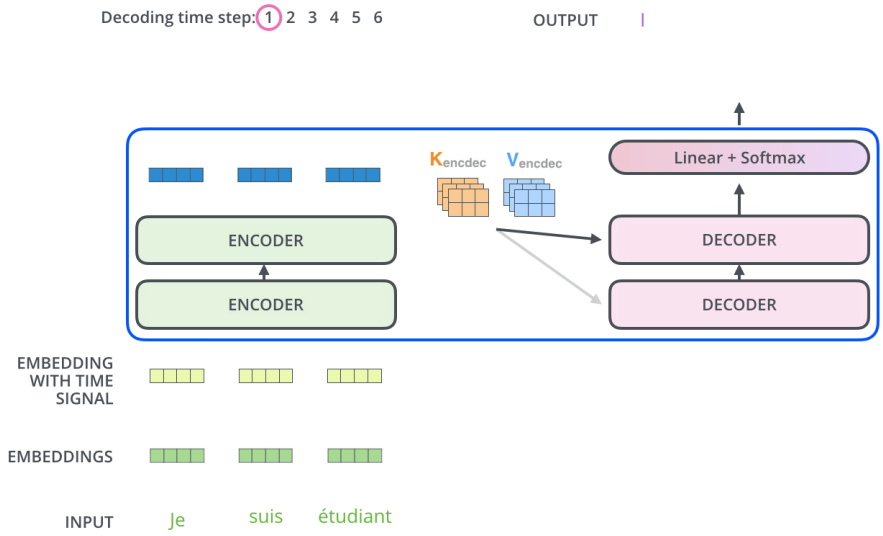

Figure 8d — Decoder emits [END]; generation stops.

Step 9 — Decoder Internals: Three Sub-layers

Sub-layer	Q / K / V source	Purpose
① Masked Multi-Head Self-Attention	Q=K=V = decoder's own representations	Each output position attends to all earlier positions. Causal mask prevents attending to future tokens (i cannot attend j>i).
② Multi-Head Cross-Attention	Q from decoder; K,V from encoder output	Connects decoder to encoder. Each decoder position attends to all encoder positions — how the model 'reads' the source.
③ Feed-Forward Network	Per-position	Same FFN as Encoder (d_model→d_ff→d_model with ReLU).

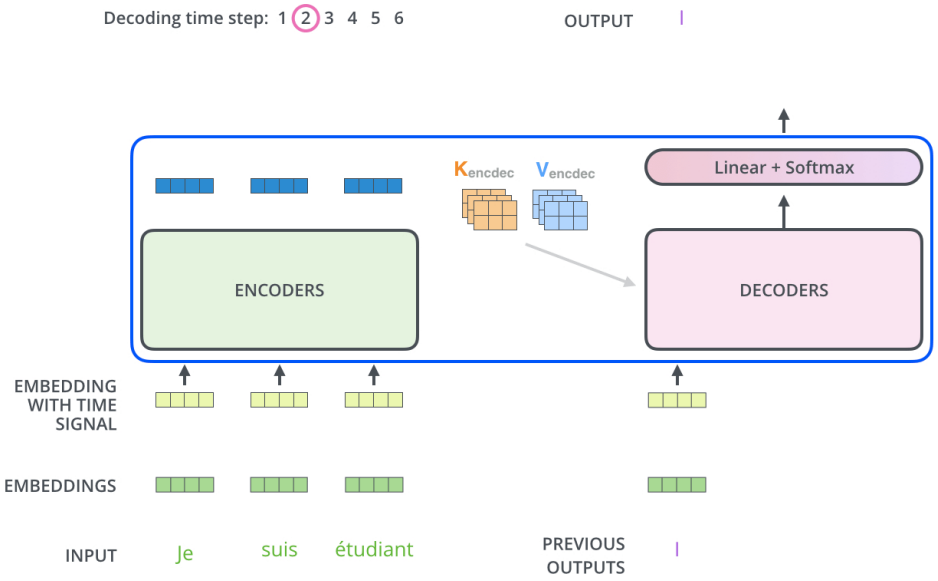

Figure 9a — Cross-attention detail. K_encdec/V_encdec (orange/blue stacks) are produced once by the Encoder and passed into every Decoder layer at every step.

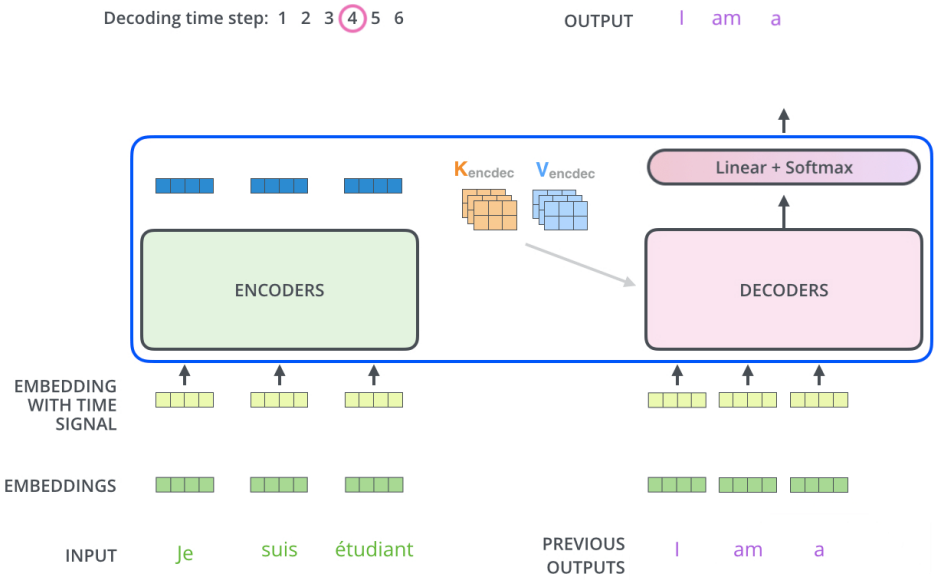

Figure 9b — As decoding progresses, decoder input grows (all previous outputs accumulated). Masked self-attention attends only to positions 0…t–1.

Step 10 — Output Projection: Linear + Softmax

What is happening?

The final Decoder vector (d_model) is projected to vocabulary size V via a Linear layer, then Softmax converts logits to probabilities. argmax selects the next token.

$$p = \mathrm{softmax}(h\,W_{\mathrm{out}}), \quad \hat{y} = \arg\max_{v} p_v$$

Symbol	Shape	Meaning
Decoder output	1 × d_model	Hidden state for current time step
W_out	d_model × V	Linear projection weight (V = vocab size)
logits	1 × V	Raw unnormalised scores per token
p = softmax	1 × V	Probability distribution over vocabulary
argmax	scalar	Index of highest-probability token

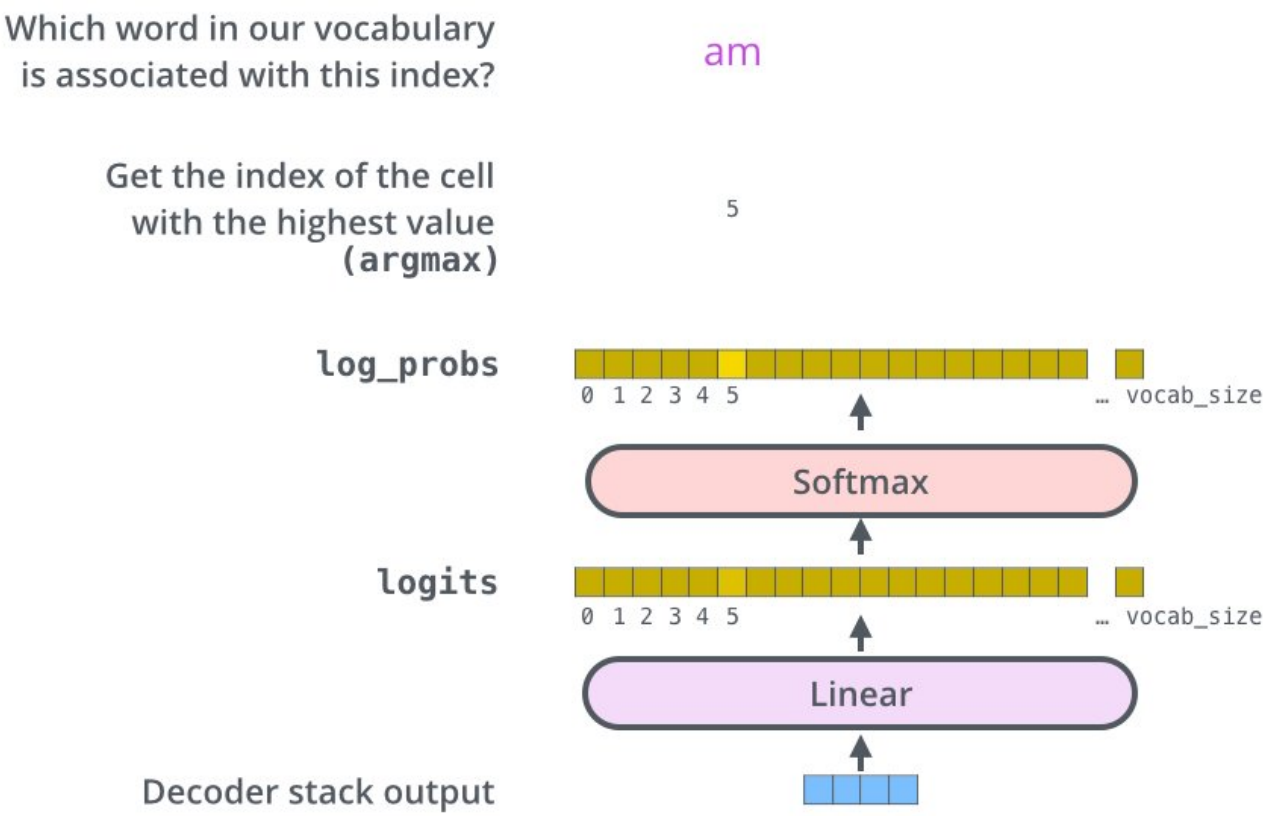

Figure 10 — Decoder stack output → Linear → logits (size vocab_size) → Softmax → log_probs → argmax picks token index (e.g. 5 → 'am').

Step 11 — Training the Transformer

Overview

During training the full source-target pair is known. **Teacher Forcing** feeds the ground-truth target (shifted right) as decoder input at every step — stabilises training but introduces exposure bias at inference (model sees its own outputs, not ground-truth).

Loss Function — Cross-Entropy

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T}\sum_{v=1}^{V} y_{t,v} \log p_{t,v}$$

Symbol	Meaning
T	Number of target tokens
V	Vocabulary size
y_{t,v}	One-hot ground-truth: 1 if token v is correct at step t, else 0
p_{t,v}	Model's predicted probability for token v at step t (softmax output)

Label Smoothing (ε=0.1): instead of hard one-hot targets, 0.9 on the correct token and 0.1/V spread over all others. Improves generalisation and BLEU.

Optimiser — Adam with Warm-up Learning Rate Schedule

$$\mathrm{lr}(\mathrm{step}) = d_{\mathrm{model}}^{-0.5} \cdot \min\left(\mathrm{step}^{-0.5},\ \mathrm{step} \cdot \mathrm{warmup}^{-1.5}\right)$$

Hyper-parameter	Value	Meaning
Optimiser	Adam	β1=0.9, β2=0.98, ε=1e-9
warmup_steps	4000	LR rises linearly for first 4000 steps
LR schedule	Warm-up→decay	Avoids large updates before embeddings stabilise; then decays ∝ 1/√step
d_model	512	Controls the LR peak magnitude

Regularisation

Technique	Value	Purpose
Residual Dropout	p=0.1	Applied to each sub-layer output before Add&Norm;
Attention Dropout	p=0.1	Applied to attention weights after softmax
Label Smoothing	ε=0.1	Smooths one-hot targets; reduces overconfidence

Training Infrastructure (paper)

Detail	Value
Hardware	8× NVIDIA P100 GPUs
Base model	100,000 steps (~12 hours)
Big model	300,000 steps (~3.5 days)
Batch size	~25,000 source + ~25,000 target tokens per batch

Training Loop (pseudocode)

```
for each batch (src, tgt):
  tgt_in = [START] + tgt[:-1] # teacher-forcing input
  tgt_out = tgt # ground-truth labels
  logits = Transformer(src, tgt_in) # forward pass
  loss = CrossEntropy(logits, tgt_out, label_smoothing=0.1)
  loss.backward() # backpropagation
  optimizer.step() # Adam parameter update
  lr = warmup_schedule(step) # update learning rate
```